

1. 다중 에이전트 시스템(MAS)으로의 패러다임 전환

인공지능(AI) 기술이 발전함에 따라, 단일 AI 에이전트/모델 이 모든 문제를 해결하는 모놀리식(monolithic) 접근 방식은 한계에 부딪히고 있습니다. 복잡하고 동적인 실제 세계의 문제들은 여러 전문화된 주체들의 협력을 통해 더 효과적으로 해결될 수 있습니다.

이러한 배경에서 다중 에이전트 시스템(Multi-Agent Systems, MAS)은 차세대 AI 애플리케이션의 핵심 아키텍처로 부상하고 있습니다.



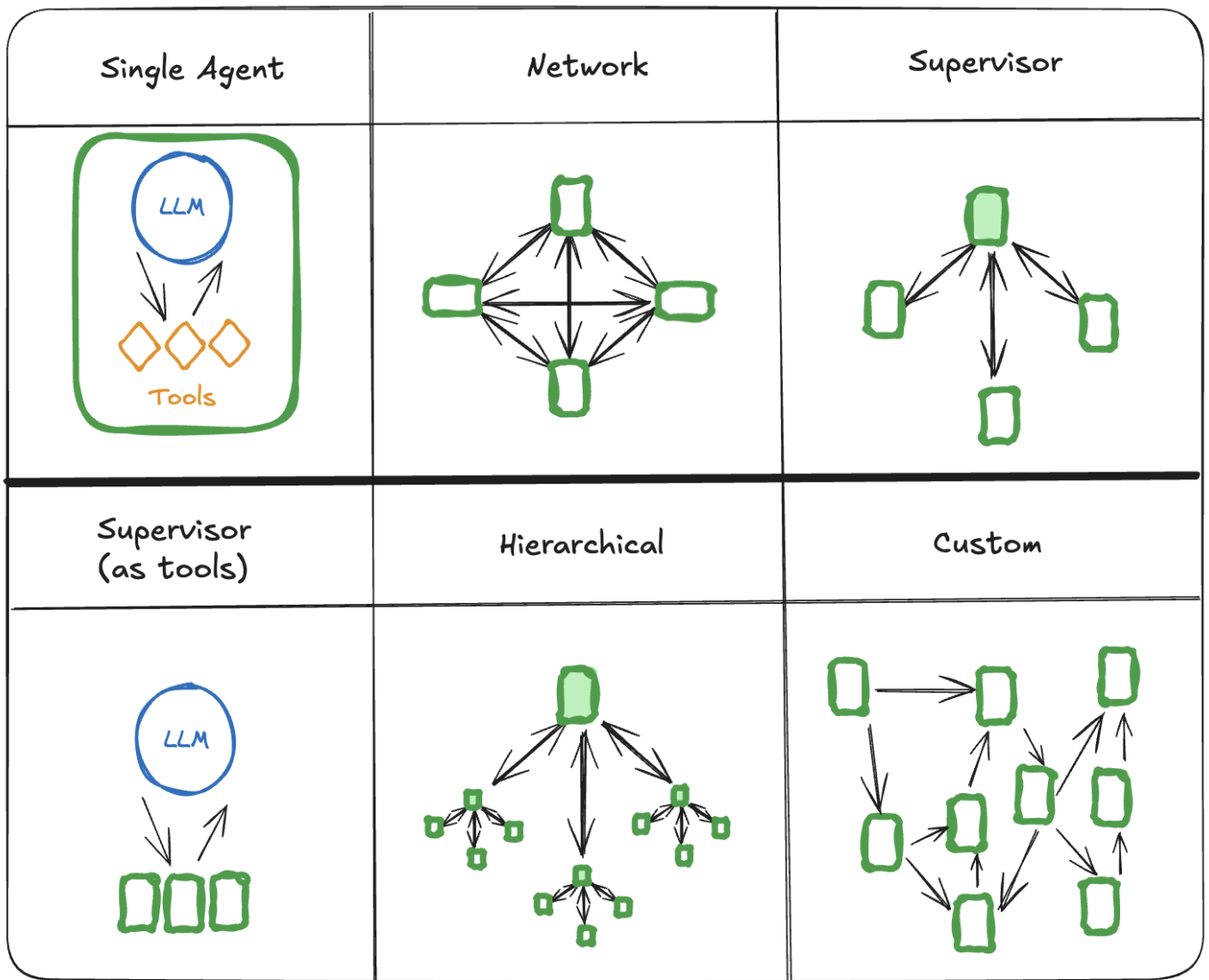
MAS를 구성하는 핵심 원칙은 다음과 같습니다.

- **자율성(Autonomy):** 에이전트는 독립적으로 작동하며, 자체적인 계획, 추론, 도구 호출 능력을 갖추고 외부의 직접적인 개입 없이 자신의 상태를 제어하고 작업을 수행합니다.⁴
- **전문화(Specialization):** 각 에이전트는 특정 도메인이나 기능에 맞게 미세 조정되거나 특수 목적으로 구축될 수 있습니다. 이는 시스템 전체의 효율성과 견고성을 크게 향상시킵니다.⁵ 예를 들어, 데이터 분석 에이전트, 사용자 인터페이스 에이전트, 외부 API 연동 에이전트 등으로 역할을 분담할 수 있습니다.
- **협력 및 조정(Collaboration & Coordination):** 에이전트들은 공동의 목표를 달성하기 위해 직접적 또는 간접적으로 통신하고 행동을 조정합니다. 이를 통해 상호 의존성을 관리하고 복잡한 작업을 분산 처리합니다.³ 이는 단순히 다른 에이전트를 '도구'로 호출하는 단일 에이전트 시스템의 방식과는 근본적으로 구별되는 특징입니다.

기본적인 협력 아키텍처

MAS 내에서 에이전트들이 상호작용하는 방식은 크게 세 가지 아키텍처로 구분할 수 있습니다. 각 구조는 시스템의 복잡성, 확장성, 그리고 제어 방식에 영향을 미칩니다.

- **중앙 집중형(Orchestrated) 네트워크:** 단일 서버나 '오케스트레이터' 에이전트가 상호작용과 정보 흐름을 통제하는 구조로, 프로젝트 매니저와 유사한 역할을 합니다. 이 모델은 통신을 단순화하고 표준화하기 용이하지만, 중앙 제어 지점이 단일 장애점(Single Point of Failure)이 될 수 있는 단점이 있습니다. Google의 ADK가 채택한 계층적 구조는 이 모델에 기반을 두고 있습니다.
- **분산형(Choreographed) 네트워크:** 에이전트들이 공동의 목표에 대한 이해를 바탕으로 중앙 제어 없이 직접 상호작용을 관리합니다. 이 모델은 중앙 집중형보다 견고하고 확장성이 뛰어나지만, 에이전트 간의 조정을 위한 더 복잡한 프로토콜을 요구합니다. 이는 서로 다른 생태계의 에이전트들이 중앙 컨트롤러 없이 협력하는 A2A 프로토콜의 비전과 맞닿아 있습니다.
- **계층형(Holonic) 시스템:** 중앙 집중형과 분산형의 하이브리드 모델로, 하나의 리더 에이전트가 여러 하위 에이전트를 포함하여 외부에서는 단일 개체('홀론')처럼 보이지만, 내부적으로는 협력적인 구조를 가집니다. 이 구조는 복잡성을 효과적으로 관리하면서 모듈성을 유지할 수 있으며, Google ADK가 명시적으로 채택하고 있는 설계 패턴입니다.



2. Agent2Agent(A2A) 프로토콜

지금까지 에이전트를 '구축'하는 프레임워크에 초점을 맞추었다면, 본 장에서는 구축된 에이전트들을 '연결'하는 기술로 시선을 전환합니다. Agent2Agent(A2A) 프로토콜은 특정 개발 프레임워크가 아니라, 이기종 에이전트 생태계 간의 원활한 소통을 위한 개방형 '통신 표준'입니다.

2.1 프레임워크가 아닌 프로토콜

A2A는 '에이전트 사일로(agent silo)' 문제를 해결하기 위해 등장했습니다. ADK, LangGraph, CrewAI 등 서로 다른 프레임워크로 만들어진 에이전트들은 복잡한 맞춤형 통합 없이는 서로 통신할 수 없습니다.

A2A의 목표는 이러한 이질적인 에이전트들이 서로의 내부 로직, 메모리, 도구를 노출하지 않고도(불투명성, opacity 원칙) 서로를 발견하고, 상호작용 방식을 협상하며, 협력할 수 있도록 하는 개방형 범용 통신 표준, 즉 'AI 에이전트를 위한 USB'가 되는 것입니다. 이 '불투명성' 원칙은 각 기업이 자체 개발한 독점적인 기술이나 로직을 보호하면서도 외부 에이전트와 협력할 수 있게 해주는 핵심적인 특징입니다.

2.2. 에이전트와 신원 (Agent & Identity)

- Agent (에이전트)
A2A 네트워크의 기본 참여 단위로, 특정 기술(Skill)을 가지고 작업을 수행할 수 있는 자율적인 소프트웨어입니다. 사람이나 다른 에이전트의 요청을 받아 처리합니다.
- Agent Card (에이전트 카드)
에이전트의 "디지털 명함" 또는 "자기소개서"입니다. 다른 에이전트가 해당 에이전트를 발견하고 그 능력을 이해하는 데 필요한 모든 정보(이름, 설명, URL, 기술 목록 등)가 agent.json 파일 형식으로 담겨 있습니다.

에이전트 카드 명세

에이전트 카드: 디지털 명함 📇

```
"name": "Travel Agent",
"description": "Plans trips...",
"url": "https://.../a2a",
"skills": [
  { "id": "book_flight", ... }
]
```

에이전트는 이 카드를 통해 자신의 정체성, 능력, 소통 방법을 외부에 알립니다.

에이전트 카드는 A2A 프로토콜의 발견 메커니즘의 핵심으로, 에이전트의 '디지털 명함' 역할을 합니다. 이는 일반적으로 웹 서버의 `/.well-known/agent.json` 경로에 위치하는 기계가 읽을 수 있는 JSON 문서로, 에이전트의 신원, 기능, 연결 세부 정보를 외부에 알립니다.

에이전트 카드의 구조는 다음과 같은 주요 필드로 구성됩니다.

필드명	데이터 타입	설명	예시
<code>name</code>	String	사람이 읽을 수 있는 에이전트의 이름입니다.	"Hello World Agent"
<code>description</code>	String	에이전트가 수행하는 작업에 대한 간략한 설명입니다.	"Just a hello world agent"
<code>version</code>	String	에이전트의 버전 정보입니다.	"1.0.0"
<code>url</code>	String	A2A 서비스를 위한 에이전트의 기본 엔드포인트 URL입니다.	"http://localhost:9999/"
<code>capabilities</code>	Object	스트리밍, 푸시 알림 등 지원하는 A2A 기능들을 명시합니다.	<code>{"streaming": true}</code>
<code>skills</code>	Array of Objects	에이전트가 제공하는 기술(Skill)들의 목록입니다.	<code>[{"id": "hello_world", ...}]</code>
<code>skills.id</code>	String	기술의 고유 식별자입니다.	"hello_world"
<code>skills.name</code>	String	기술의 이름입니다.	"Returns hello world"
<code>skills.inputModes</code>	Array of Strings	기술이 지원하는 입력 MIME 타입입니다.	<code>["text/plain"]</code>
<code>authentication</code>	Object	클라이언트가 에이전트와 통신할 때 필요한 인증 스킴을 정의합니다.	<code>{"schemes": ["public"]}</code>

이 상세한 구조는 클라이언트 에이전트가 다른 에이전트의 정체성, 상호작용 방법, 그리고 수행 가능한 특정 작업을 프로그래밍 방식으로 이해할 수 있게 해줍니다.

2.3. 에이전트의 능력 (Abilities)

- Tool (도구)**
가장 구체적인 실행 단위로, 코드상의 특정 함수에 해당합니다. 예를 들어, `add(a, b)` 함수는 하나의 Tool입니다. 이는 에이전트가 수행할 수 있는 가장 작은 원자적 작업입니다.
- Skill (기술)**
사용자 관점에서 에이전트가 해결할 수 있는 의미 있는 작업 단위입니다. 하나 이상의 Tool을 조합하여 구성될 수 있습니다. 예를 들어, '항공권 예약'이라는 Skill은 '날짜 조회', '좌석 선택', '결제 처리'라는 여러 Tool들로 이루어질 수 있습니다.
- Capability (역량)**
에이전트가 '무엇을' 하는지(Skill)가 아니라 '어떻게' 작동하는지에 대한 기술적 특성입니다. 예를 들어, `streaming: true`는 이 에이전트가 결과를 스트리밍 방식으로 점진적으로 전송할 수 있는 Capability를 가졌음을 의미합니다.

2.4. 상호작용과 통신 (Interaction & Communication)

- Task (작업)**
한 에이전트(클라이언트)가 다른 에이전트(서버)에게 요청하는 구체적인 업무의 단위입니다. "5와 3을 더해줘"라는 요청은 하나의 Task가 됩니다. 이 Task는 고유한 ID를 가지며, `submitted`, `working`, `completed`, `failed` 같은 생명주기(Lifecycle) 상태를 가집니다.
- Event (이벤트)**
Task가 진행되는 동안 발생하는 모든 상태 변화나 데이터 교환을 의미합니다. 예를 들어, Task가 성공적으로 제출되었음(`task_submitted`), 작업이 시작되었음(`status_update: working`), 최종 결과가 나옴(`result`) 등이 모두 개별 Event입니다.
- Message (메시지)**
에이전트 간에 교환되는 데이터의 구조화된 단위입니다. 하나의 Message는 여러 개의 Part로 구성될 수 있습니다.
- Part (부분)**
Message를 구성하는 개별 데이터 조각입니다. 텍스트는 `TextPart`, JSON 데이터는 `DataPart`, 이미지는 `ImagePart` 등 다양한 유형의 데이터를 담을 수 있습니다.

2.5 작업 수명 주기 및 메시징

A2A는 기존의 검증된 웹 표준 위에 구축되어 있어, 현재의 기술 스택에 쉽게 통합될 수 있습니다.

- 전송 계층(Transport Layer):** 통신은 HTTP(S)를 기반으로 하며, 원격 프로시저 호출(RPC)을 위해 경량의 JSON-RPC 2.0 형식을 사용합니다. 실시간 스트리밍 통신을 위해서는 웹소켓보다 구현이 간단하고 방화벽 친화적인 서버-전송 이벤트(Server-Sent Events, SSE)를 채택했습니다.
- 작업 관리(Task Management):** A2A 내의 모든 상호작용은 고유 ID를 가진 '작업(Task)'으로 관리됩니다. 작업은 `submitted` (제출됨), `working` (진행 중), `input_required` (입력 필요), `completed` (완료), `failed` (실패), `cancelled` (취소)와 같은 명확한 수명 주기 상태를 가집니다.
- 통신 메서드:**

- 동기식 (`message/send`): 빠른 작업이나, 클라이언트가 `tasks/get` 메서드를 통해 주기적으로 상태를 폴링(polling)하려는 경우에 사용됩니다.
- 스트리밍 (`message/stream`): 오래 실행되는 작업에 대해 SSE를 사용하여 실시간으로 서버에서 클라이언트로 상태 업데이트를 전송합니다.

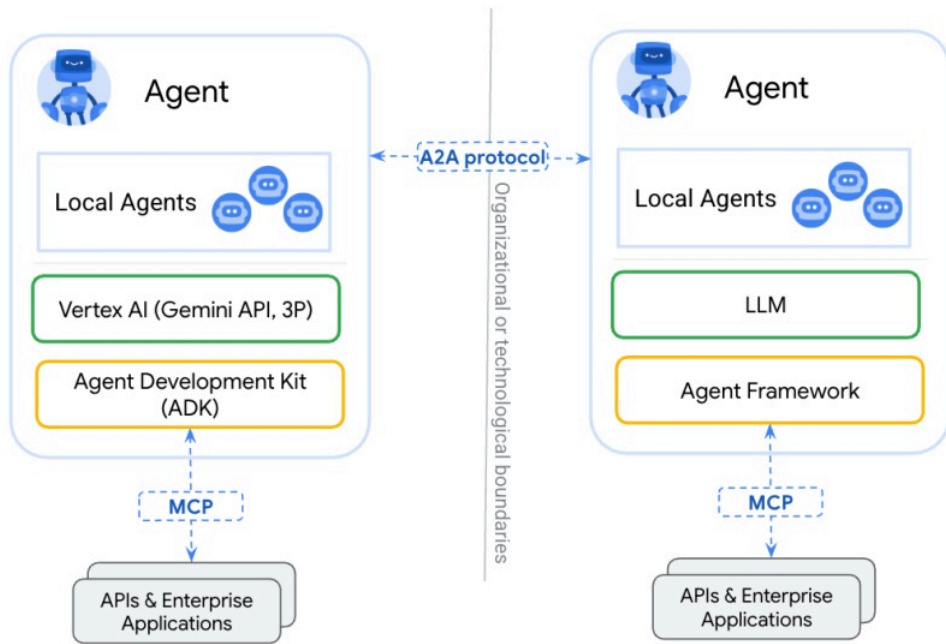
2.6 A2A와 MCP

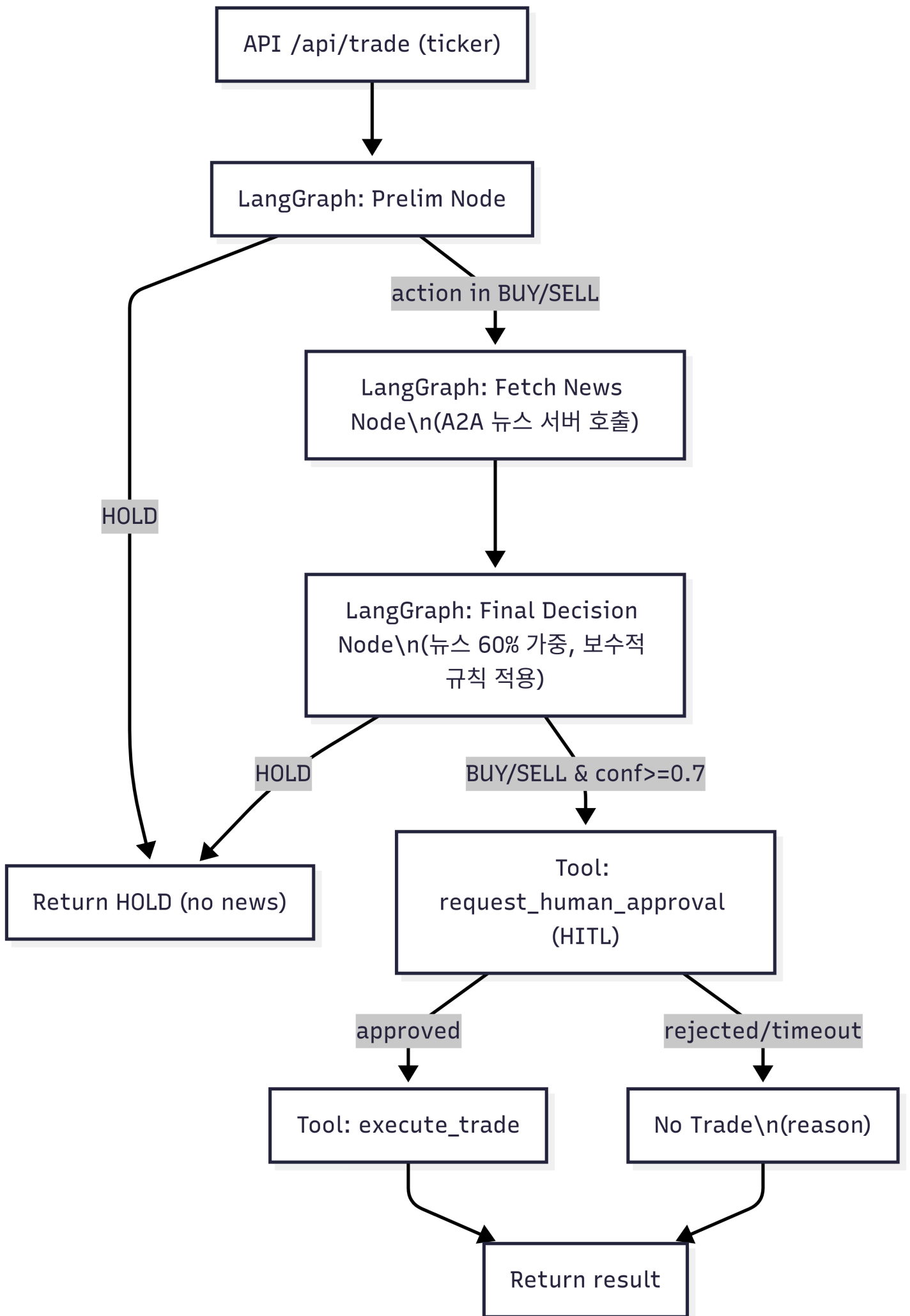
A2A 프로토콜을 이해할 때, 모델 컨텍스트 프로토콜(Model Context Protocol, MCP)과의 관계를 명확히 하는 것이 중요합니다. 이 두 프로토콜은 경쟁 관계가 아니라, 서로 다른 문제를 해결하는 상호 보완적인 관계에 있습니다.

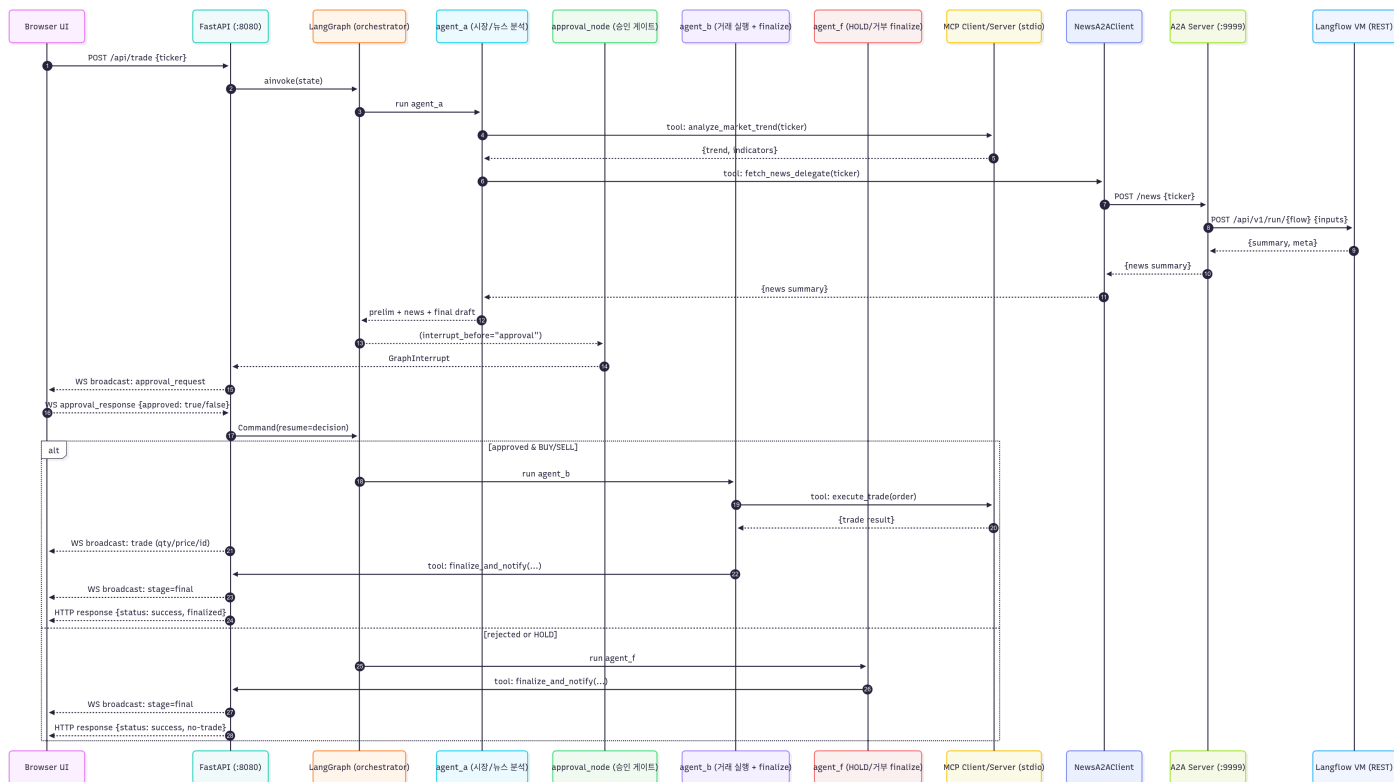
특징	Agent2Agent (A2A)	Model Context Protocol (MCP)
주요 사용 사례	에이전트 간(Agent-to-Agent) 협력	에이전트-도구 간(Agent-to-Tool) 연결
통신 패러다임	작업 기반, 대화형, 비동기 통신	안전한 데이터 및 도구(API) 접근
주도 기관	Google 및 50+ 파트너	Anthropic 및 파트너

실제 시나리오에서 에이전트는 먼저 MCP를 사용하여 외부 데이터베이스나 API(도구)로부터 데이터를 안전하게 수집한 다음, A2A 프로토콜을 사용하여 그 데이터를 분석이나 추가 처리를 위해 다른 전문 에이전트에게 전달할 수 있습니다.

✓ (이후 실습으로 대체) 함께 코드 살펴보기







ChatGPT - 병합된 종속성 목록

새 채팅

채팅 검색

라이브러리

Codex

Sora

GPT

Cello Square - Logistics Se...

[DEV] Cello Square - Logis...

Translation Agent

Cello Square - Logistics M...

[DEV] 마케팅 인덱스

새 프로젝트

It Takes Two

P

SDEV

wltsrnl

일상

더 보기

채팅

병합된 종속성 목록

Uv sync 락 생성 확인

영어로 번역

Key AI updates for your strategy

Replit 여러 포트 띄우기

코드 분석 및 설명

MCP 서버 설정 확인

React 플로우 이미지 찾기

클라이언트 스트리밍 설정

A2A 구현 방법

MCP 서버 등록 문제

세션 문제 원인 분석

Google A2A SDK 조사

MCP 서버 문제 분석

MCP 클라이언트 예제

MCP 서버 예제 분석

Replit 용량 문제 해결

HITL 거래 봇 수정

Tool call error 해결

기능 개선 제안

Reactive programming 설명

교육자료 작성 요청

hanyata awithi

ChatGPT 5

공유하기

코드 복사

```
web_thread = threading.Thread(target=run_web_server, args=(url,))
web_thread.start()

while True:
    await asyncio.sleep(1)

except KeyboardInterrupt:
    print("\n🔴 종료 요청")
except Exception as e:
    print(f"\n❌ 시스템 오류: {e}")
    import traceback
    traceback.print_exc()
finally:
    print("\n🔴 시스템 종료됨")

if __name__ == "__main__":
    asyncio.run(main())
```

메모/적용 포인트

- session_7_mcp_server.py 파일명을 위 코드가 참조하므로(같은 폴더) 그대로 두거나 이름 맞춰줘.
- 환경변수:
 - OPENAI_API_KEY (필수)
 - A2A_BASE_URL (없으면 http://127.0.0.1:9999)
 - A2A_SERVER_PORT (기본 9999)
 - PORT (Replit 외부 노출용, 기본 8080)
- /api/trade 호출 시 그래프가 스트리밍으로 돌면서 WS에 단계 이벤트를 쏜다(agent_a, agent_b, finalize_only start/done).
- 승인(HITL)은 기존과 동일하게 톨 내부에서 대기하는 POC 방식(간단/안전). 후속 단계에서 "승인 도착 시 동일 thread_id로 재호출(resume)" 패턴으로 바꾸고 싶으면 안내할게.

출처

좋은데 2가지 관점에서 알아보고 적용해줘 휴먼 어프로치는 interrupt와 command로 랭그래프는 hitl 구현을 지원한다.

이 부분 하고, 톨 스키마를 지정하면 시스템프롬프트에 json 형태를 다치지 않아도 된다.

이 두가지 랭그래프 사용법 조사한뒤에 다시 전체 코드를 제공해줘

GPT-5 Pro 요청 효과적으로 구현하기...

세부 정보

+ 무엇이든 물어보세요

ChatGPT는 실수를 할 수 있습니다. OpenAI에서는 Awithi 워크스페이스 데이터를 이용해 모델을 훈련하지 않습니다.

세부 정보

톨 스키마를 지정하면 시스템 프롬프트에 JSON 형태를 포함하지 않아도 된다. create_react_agent는 자동으로 톨 설명을 포함하고, 최종 출력 형상을 pydantic 또는 JSON 스키마로 강제할 수 있다.

구현 중 도구 적재

다양한 도구를 MultiServerMCPClient로 연결하고 있으며, structured output을 사용해 명령을 구조화하여 정리하고 있어.

도구 호출 관리

request_human_approval 도구를 구현하면서, WebSocket 대신 interrupt를 활용해 허가 독립적인 인터페이스를 구성하고 있어.

검토 증명

JavaScript의 entrypoint와 동일한 개념을 다루고 있어. Python 문서에서는 create_react_agent 및 interrupt_before와 관련된 부분을 살펴보고 있어.

Stream 처리와 실제 시간 제어

Http POST /api/trade에서 graph.stream()로 스트림을 처리하고, 중단과 승인 후 재개하는 방식으로 이동하고 있어.

가장 탐색하기

내가 살고 있는 집 주변을 걸어 보니, 낯익은 집들이 눈에 띄고 있네. 집마다 꽃밭이 아름답고 사람들이 정원을 가꾸고 있네. 기분이 좋아지고 이 아름다움을 감사하게 여겨.

소풍 지나기

아이들과 함께 소풍을 떠나고 있어. 푸른 하늘 아래 펼쳐진 잔디밭에서 놀고, 맛있는 도시락을 먹으니 행복이 넘쳐나고 있어.

고양이와 시간 보내기

부엌에서 요리하는 동안 고양이와 내발에 올라앉아 있어. 부드러운 털을 자주 쓰다듬어 주고 있어. 이 부드러운 순간들이 정말 소중해.

도구 함수 설명

분석 및 뉴스 가져오기 도구를 다루고 있어. 요청 승인 도구는 외부 호출을 중지하고, 앱에서 사용자 승인을 확인 후 다음 단계로 넘어가고 있어.

활성화된 도구 평가

도구 호출을 위한 request_human_approval 도구를 평가하고 있어. 승인되지 않으면 execute_trade 호출을 중단하고 있어. request_human_approval은 중단 이벤트로 작동 중이야.

작동하는 구조 살펴보기

create_react_agent의 전원을 고려하고 있어. interrupt_before=['tools']를 활용해 도구 실행 직전과 직후를 관리하고, AI 메시지 상태를 조정하여 도구 실행을 조절 중이야.

편집하고 있어

그래프 상태를 보고 있는 중이야.

명령어 사용하기

Command python으로 그래프 상태를 바꾸고 있어 요.

제작자 조정

커스텀 하지만 복잡한 LangGraph를 설계하고 있어. Node "agent"는 ChatOpenAI로 도구를 활용 중이야.

효과적으로 구현하기

request_human_approval을 interrupt로 구현하고, UI 이벤트를 외부에서 처리하며 JSON을 반환하는 중입니다.

8 / 8
Chapter 6-7. A2A 개념과 적용 | exported: 25. 11. 24. 오전 8:19